

1. Primero revisemos 23C

Ejecuta este diagnóstico sobre:

bu_frd.model_party_behavior_windows_2022_2025

SELECT

COUNT(*) AS parties,

```
/* =====  
MONTH AVAILABILITY MISMATCH  
===== */
```

```
SUM(  
  CASE  
    WHEN utilization_recent_months_available  
      <> weighted_util_recent_months_available  
    THEN 1 ELSE 0  
  END  
) AS recent_month_count_mismatch,
```

```
SUM(  
  CASE  
    WHEN utilization_prior_months_available  
      <> weighted_util_prior_months_available  
    THEN 1 ELSE 0  
  END  
) AS prior_month_count_mismatch,
```

```
/* =====  
RECENT VALUE NULL MISMATCH  
===== */
```

```
SUM(  
  CASE  
    WHEN  
      (  
        avg_utilization_recent_3m IS NULL  
        AND weighted_utilization_recent_3m IS NOT NULL  
      )  
    OR  
      (  
        avg_utilization_recent_3m IS NOT NULL  
        AND weighted_utilization_recent_3m IS NULL  
      )  
  END  
)
```

```

        THEN 1 ELSE 0
    END
) AS recent_value_null_mismatch,

/* =====
PRIOR VALUE NULL MISMATCH
===== */

SUM(
    CASE
        WHEN
            (
                avg_utilization_prior_9m IS NULL
                AND weighted_utilization_prior_9m IS NOT NULL
            )
        OR
            (
                avg_utilization_prior_9m IS NOT NULL
                AND weighted_utilization_prior_9m IS NULL
            )
        THEN 1 ELSE 0
    END
) AS prior_value_null_mismatch

FROM
    bu_frd.model_party_behavior_windows_2022_2025;

```

Lo esperado

Los cuatro deben ser:

```

recent_month_count_mismatch = 0
prior_month_count_mismatch  = 0
recent_value_null_mismatch  = 0
prior_value_null_mismatch   = 0

```

Si alguno no es cero, sabemos que **23C no se reconstruyó correctamente con la tabla corregida**.

2. Quiero además ver exactamente dónde están esos 4,096

Ejecutemos:

```

SELECT

    model_split,
    label_3m,

    COUNT(*) AS parties,

    SUM(
        CASE
            WHEN avg_utilization_recent_3m IS NULL
            THEN 1 ELSE 0
        END
    ) AS avg_recent_missing,

    SUM(
        CASE
            WHEN weighted_utilization_recent_3m IS NULL
            THEN 1 ELSE 0
        END
    ) AS weighted_recent_missing,

    SUM(
        CASE
            WHEN avg_utilization_prior_9m IS NULL
            THEN 1 ELSE 0
        END
    ) AS avg_prior_missing,

    SUM(
        CASE
            WHEN weighted_utilization_prior_9m IS NULL
            THEN 1 ELSE 0
        END
    ) AS weighted_prior_missing

FROM
    bu_frd.model_party_behavior_windows_2022_2025

GROUP BY
    model_split,
    label_3m

ORDER BY
    model_split,
    label_3m DESC;

```

Esto nos dirá si la diferencia proviene principalmente de:

RECENT 3M

o de:

PRIOR 9M

3. También podemos probar directamente la lógica matemática

Para que:

avg_utilization_recent_3m
-
avg_utilization_prior_9m

sea válido, necesitamos:

recent != NULL
AND
prior != NULL

Lo mismo debe ocurrir para weighted.

Entonces ejecuta:

```
SELECT

COUNT(*) AS parties,

SUM(
  CASE
    WHEN avg_utilization_recent_3m IS NOT NULL
    AND avg_utilization_prior_9m IS NOT NULL
    THEN 1 ELSE 0
  END
) AS avg_util_change_available,

SUM(
  CASE
    WHEN weighted_utilization_recent_3m IS NOT NULL
    AND weighted_utilization_prior_9m IS NOT NULL
    THEN 1 ELSE 0
  END
) AS weighted_util_change_available,

SUM(
  CASE
```

```

        WHEN
            (
                avg_utilization_recent_3m IS NOT NULL
                AND avg_utilization_prior_9m IS NOT NULL
            )
        <>
            (
                weighted_utilization_recent_3m IS NOT NULL
                AND weighted_utilization_prior_9m IS NOT NULL
            )
        THEN 1 ELSE 0
    END
) AS change_availability_mismatch

```

```

FROM
    bu_frd.model_party_behavior_windows_2022_2025;

```

Si Spark no acepta <> entre expresiones booleanas, usa esta versión:

```

SELECT

    COUNT(*) AS parties,

    SUM(
        CASE
            WHEN avg_utilization_recent_3m IS NOT NULL
            AND avg_utilization_prior_9m IS NOT NULL
            THEN 1 ELSE 0
        END
    ) AS avg_util_change_available,

    SUM(
        CASE
            WHEN weighted_utilization_recent_3m IS NOT NULL
            AND weighted_utilization_prior_9m IS NOT NULL
            THEN 1 ELSE 0
        END
    ) AS weighted_util_change_available,

    SUM(
        CASE
            WHEN
                (
                    avg_utilization_recent_3m IS NOT NULL
                    AND avg_utilization_prior_9m IS NOT NULL
                    AND
                    (
                        weighted_utilization_recent_3m IS NULL

```

```

        OR weighted_utilization_prior_9m IS NULL
    )
)

OR

(
    weighted_utilization_recent_3m IS NOT NULL
    AND weighted_utilization_prior_9m IS NOT NULL
    AND
    (
        avg_utilization_recent_3m IS NULL
        OR avg_utilization_prior_9m IS NULL
    )
)

THEN 1 ELSE 0
END
) AS change_availability_mismatch

```

FROM
 bu_frd.model_party_behavior_windows_2022_2025;

Aquí esperamos:

avg_util_change_available
 = 688,589 - 198,963
 = 489,626

Por lo tanto weighted también debería ser:

489,626

y:

change_availability_mismatch = 0

4. Mi sospecha principal: una tabla quedó con la versión anterior

Por la evidencia actual, esto es lo más probable.

La lógica es:

Party-month

avg availability = weighted availability



Prior/Recent aggregation



debería conservar la misma disponibilidad



Current change



debería producir el mismo missing count

No existe una razón matemática para terminar con:

198,963 vs 194,867

si 23C realmente está construido desde la nueva tabla.

Esto puede pasar fácilmente si se ejecutó:

recreate utilization monthly

pero no se volvió a ejecutar realmente uno de:

23C windows

23D current features

o si Spark está utilizando metadata/cache anterior.

5. Si el primer QA da mismatch > 0

Yo haría una reconstrucción limpia de **solo 23C y 23D**.

Primero:

REFRESH TABLE

bu_frd.model_party_utilization_monthly_2022_2025;

Si la tabla estuviera cacheada en la sesión, también:

UNCACHE TABLE IF EXISTS

bu_frd.model_party_behavior_windows_2022_2025;

UNCACHE TABLE IF EXISTS

bu_frd.model_behavior_features_current_2022_2025;

Luego:

DROP TABLE IF EXISTS

bu_frd.model_party_behavior_windows_2022_2025;

y ejecutar nuevamente el SQL completo de **23C**.

Después:

DROP TABLE IF EXISTS

bu_frd.model_behavior_features_current_2022_2025;

y volver a ejecutar **23D**.

No tenemos que tocar nada anterior.

6. Si 23C da todo cero, el problema está solamente en 23D

Si obtenemos:

recent_month_count_mismatch = 0

prior_month_count_mismatch = 0

recent_value_null_mismatch = 0

prior_value_null_mismatch = 0

entonces 23C está perfecto.

En ese escenario simplemente recreamos 23D porque la tabla:

bu_frd.model_behavior_features_current_2022_2025

quedó construida a partir de una versión anterior de windows.

Después del rebuild, este QA:

SELECT

COUNT(*) AS rows,

COUNT(DISTINCT party_id) AS parties,

SUM(

CASE

WHEN utilization_change_vs_baseline IS NULL

THEN 1 ELSE 0

END

) AS utilization_change_missing,

SUM(

```

CASE
  WHEN weighted_utilization_change_vs_baseline IS NULL
  THEN 1 ELSE 0
END
) AS weighted_util_change_missing

FROM
  bu_frd.model_behavior_features_current_2022_2025;

```

debería quedar:

rows	688,589
parties	688,589
utilization_change_missing	198,963
weighted_util_change_missing	198,963

Ese es ahora nuestro **resultado esperado exacto**.

No avanzaría todavía al Z-score hasta resolver esos 4,096, porque este tipo de discrepancia pequeña es precisamente la que conviene eliminar antes del modelado. Después de eso podemos congelar definitivamente 23C/23D y construir 23E.